ITHS 2026

Kom igång före lektion 1

Python · gör detta innan tisdagens första pass

1

Introduktion

Vem som håller kursen, och vad som ska vara klart

Välkommen

- Kursen är programmering i **Python**.
- Lärare: **Michael Granbäck**. Undervisar också i Java.
- Till vardags på **XLENT**, ett svenskt IT-bolag som även finns i Norge.

Om Python

- Används i verktyg, infrastruktur, spel, data och webb.
- Dropbox skrevs ursprungligen i Python. Delar av YouTube och Instagram också.
- Mycket av dagens AI- och maskininlärning körs på Python.
- I den här kursen: **automatisera analys av loggar och textdata** inom IT-säkerhet.
- Relativt lätt att lära sig, och ett bra första språk.

Målet före lektion 1

- När första lektionen börjar ska du kunna **köra ett Python-program** på din egen dator.
- Det är allt som krävs. Resten lär vi oss tillsammans.
- Räkna med en kväll. Mindre om allt går smidigt.
- Varje dokument slutar med **Vanliga fel**. Kolla den först om något strular.
- Fastnar du: skriv upp exakt vad du gjorde och felmeddelandet. Ta med det till lektion 1.

Innehållet följer den här ordningen

- **1.** Verktyg — vad vi installerar och varför
- **2.** Installera uv och Python — Windows **eller** Linux/Mac
- **3.** Visual Studio Code, Git, extensionerna Python och Jupyter
- **4.** Skapa kursprojektet och koppla uv till VS Code
- **5.** Få igång JupyterLab
- **6.** Skriv och kör hello world
- **7.** Läs kursens hållning till AI-verktyg

2

Verktyg

Översikt. Steg-för-steg kommer i dokumenten efter.

Operativsystem som kursen testas på

- **Windows 11.** Allt är skrivet för Windows 11. Äldre versioner rekommenderas inte.
- **Linux:** Ubuntu 22.04 LTS eller senare. 24.04 är vanligast.
- **Mac:** de macOS-versioner som fortfarande får säkerhetsuppdateringar.

uv — allt annat går genom det

- Projekt- och pakethanterare. Installerar också Python själv.
- Används för att köra program och lägga till tredjepartsmoduler.
- Python har **pip**, men den blir krånglig med flera projekt.
- Med uv kan två mappar ha olika Python och olika paket utan att krocka.
- Exempel på paket: **pytest**, som kör tester.

Python, VS Code och Jupyter

- **Python 3.13 eller senare.** Nyare går bra. Du installerar inte Python själv — uv gör det.
- **Visual Studio Code** är editorn. Extensionerna **Python** och **Jupyter**.
- En annan editor går bra om du redan har en favorit. Instruktionerna utgår från VS Code.
- **Jupyter** (filändelse .ipynb) blandar kod, text och grafer. Används löpande i kursen.
- JupyterLab ska fungera **lokalt** före lektion 1. jupyter.org/try räcker inte.

Det du inte behöver installera själv

- **Marp** — läraren gör presentationsmaterialet med det. Du hoppar över det.
- **Git** — versionshantering. Installeras tillsammans med VS Code.
- Git används på riktigt först i lektion 3.

3 A

Installera på Windows

Windows 11. Inget kräver administratör, utom långa sökvägar.

Innan du börjar

- Du installerar **uv**. uv installerar sedan **Python** åt dig.
- Allt hamnar i din användarmapp.
- Du behöver **inte** köra som administratör, förutom steg 5.

1. Öppna PowerShell

- Start, skriv **PowerShell**, tryck Enter.
- Du behöver inte ”Kör som administratör”.
- Skriv eller klistra in ett kommando i taget. Avsluta med Enter.

2. Installera uv

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- Raden hämtar uv:s officiella skript och kör det.
- uv hamnar i %USERPROFILE%\.local\bin och mappen läggs i PATH.
- Det tar någon minut.
- Alternativ: docs.astral.sh/uv/getting-started/installation/

3. Starta om PowerShell och verifiera

```
uv self version
```

- **Stäng fönstret och öppna ett nytt.** PATH slår igenom först då.
- En rad som uv 0.12.x (… windows-msvc) betyder att det lyckats.
- Versionsnumret ser annorlunda ut när du kör. Det är rätt ändå.
- ”not recognized” — se Vanliga fel.

4. Installera Python 3.13

```
uv python install 3.13
uv python list
```

- Kursen kräver **3.13 eller senare**.
- Listan ska innehålla en rad **cpython-3.13**.
- Flera versioner kan ligga sida vid sida. uv väljer per projekt.
- Du kör inte python direkt i kursen. Det går via **uv run**.

5. Långa sökvägar (rekommenderas)

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" `
  -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force
```

- Windows stoppar sökvägar vid 260 tecken. Det stör ibland Python och uv.
- Öppna PowerShell **som administratör**: högerklicka Start, Terminal (Admin).
- Starta om datorn efteråt.

Windows — vanliga fel, 1

- **uv känns inte igen** efter omstart: öppna ett helt nytt PowerShell-fönster.
- Kvarstår det: lägg till %USERPROFILE%\.local\bin i PATH (Miljövariabler), logga ut och in.
- **running scripts is disabled:** kör installationsraden exakt som i steg 2. Den har ByPass.

Windows — vanliga fel, 2

- **Nedladdning hänger eller SSL-fel**, ofta på skolnät: prova mobil hotspot.
- Bakom proxy: sätt $env:HTTPS_PROXY innan du kör raden.
- **uv python install** med nätverksfel: samma sak, kör om på ett öppnare nät.
- **Antivirus blockerar uv.exe:** tillåt filen i %USERPROFILE%\.local\bin och installera om.

3 B

Installera på Linux och Mac

Hoppa över det här avsnittet om du redan gjort Windows.

Vilket system

- **Ubuntu 22.04 LTS eller senare.** 24.04 är vanligast. Andra distributioner testas inte.
- **Mac:** versioner som fortfarande får säkerhetsuppdateringar från Apple.
- Inget kräver sudo, utom om curl saknas. Allt hamnar i ~/.local.

1. Öppna en terminal

- **Ubuntu:** Ctrl + Alt + T, eller sök efter Terminal.
- **Mac:** Cmd + Blanksteg, skriv Terminal, tryck Enter.
- Ett kommando i taget, avsluta med Enter.

2. Installera uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh

# om curl saknas
wget -qO- https://astral.sh/uv/install.sh | sh
```

- Skriptet lägger uv i ~/.local/bin och en PATH-rad i ~/.bashrc eller ~/.zshrc.
- Saknas curl, till exempel i WSL: sudo apt install curl.
- Eller använd wget-raden här under.

3. Ladda om PATH och verifiera

```
uv self version
```

- Öppna ett **nytt** terminalfönster.
- Eller kör source ~/.bashrc respektive source ~/.zshrc.
- En rad med linux-gnu eller apple-darwin betyder att det lyckats.
- command not found — se Vanliga fel.

4. Installera Python 3.13

```
uv python install 3.13
uv python list
```

- Listan ska innehålla **cpython-3.13**.
- Rör inte systemets egen python3.
- I kursen kör du allt via **uv run**.

Linux och Mac — vanliga fel

- **uv: command not found:** echo $SHELL. zsh skriver PATH-raden i ~/.zshrc, bash i ~/.bashrc. Nytt fönster.
- **curl saknas:** sudo apt install curl, eller wget-raden.
- **unzip eller tar saknas:** sudo apt install unzip tar, kör om steg 2.
- **Nätverksfel:** öppnare nät, eller export HTTPS_PROXY=… innan kommandot.
- **Mac, developer cannot be verified:** System Settings, Privacy & Security, Open Anyway. Ovanligt för CLI.

4

Visual Studio Code

Editor, Git, och två extensioner

Varför en kodeditor

- **Syntaxfärgning** — nyckelord, text och funktioner får olika färg.
- **Feldetektering** — trasig syntax markeras direkt.
- **Komplettering** — förslag medan du skriver.
- Inbyggt stöd för git, tester och felsökning rad för rad.
- Instruktionerna utgår från VS Code. En annan editor kräver att du löser stödet själv.

Förkrav

- uv och Python ska redan vara installerade (Windows- eller Linux/Mac-avsnittet).
- De grafiska klicken är likadana på Windows, Linux och Mac.
- Terminalkommandon som skiljer sig står uppdelade.

1. Installera Visual Studio Code

```
# Ubuntu, i mappen där .deb-filen ligger
sudo apt install ./code_*.deb
```

- Ladda ner från **code.visualstudio.com**. Knappen känner av operativsystemet.
- **Windows:** kör installationsprogrammet. Låt **Add to PATH** vara ikryssad.
- **Mac:** dra Visual Studio Code till Applications.
- **Ubuntu:** sudo apt install ./code_*.deb, eller sudo snap install --classic code.
- Starta VS Code och låt det stå öppet.

2. Installera Git

```
# Ubuntu
sudo apt install git

# Mac
xcode-select --install
```

- VS Code använder Git. Vi använder det på riktigt i lektion 3. Installera nu.
- **Windows:** git-scm.com/download/win. Klicka igenom med förvalen.
- **Ubuntu:** sudo apt install git.
- **Mac:** xcode-select --install, eller ladda ner från git-scm.com.

3. Extensioner

- Extensions-ikonen i vänsterkanten (fyra rutor), eller **Ctrl + Shift + X**.
- Mac: **Cmd + Shift + X**.
- Installera **Python** från Microsoft.
- Installera **Jupyter** från Microsoft.
- Pylance och Python Debugger följer med Python-extensionen. Kontrollera att de står som installerade.

4. Stäng av AI-komplettering

- Första starten vill ofta slå på **Copilot**, som skriver kod åt dig.
- Som nybörjare blir det förvirrande. Du behöver känna vad du klarar själv.
- **Logga inte in** på Copilot när VS Code frågar.
- Har förslag redan dykt upp: **Ctrl + ,**, sök inline suggest, stäng av **Editor: Inline Suggest: Enabled**.
- Vi tar upp AI-verktyg igen sist i den här genomgången, och på lektion 1.

Verifiera editorn

```
code --version
git --version
```

- Terminal, New Terminal, inne i VS Code.
- Båda kommandona ska skriva ut ett versionsnummer.
- code saknas på Windows: starta om VS Code, och vid behov datorn, så PATH läses om.
- I Extensions ska Python och Jupyter ligga under Installed.

VS Code — vanliga fel

- **code känns inte igen:** Add to PATH missades. Windows: installera om. Mac: Command Palette, Shell Command: Install code in PATH.
- **git känns inte igen:** gör om steg 2 och starta om terminalen.
- **Extensioner syns inte:** Ctrl + Shift + P, Developer: Reload Window.
- **Copilot föreslår ändå:** steg 4, och logga ut ur Copilot.

Andra editorer

- Det går, men du löser Python- och Jupyter-stöd själv.
- Sublime Text, JetBrains PyCharm, Zed.
- I terminalen: Neovim, Emacs, Helix. De kräver mer konfiguration.

5

Skapa kursprojektet

Samma kommandon på Windows, Linux och Mac

Förkrav

- uv self version skriver ut en versionsrad.
- VS Code har extensionerna Python och Jupyter.
- Varje del av inlämningsuppgiften är senare ett eget uv-projekt. Samma arbetssätt.

Vad ett projekt är

- En mapp med **pyproject.toml**: vilken Python, vilka paket.
- uv bygger en egen miljö, **.venv**, för just den mappen.
- Två projekt kan ha olika Python och olika paket utan att krocka.

1. Skapa projektmappen

```
# Windows
mkdir C:\Kod\Pythonkurs
cd C:\Kod\Pythonkurs

# Linux / Mac
mkdir -p ~/kod/pythonkurs
cd ~/kod/pythonkurs
```

- PowerShell på Windows, valfri terminal på Linux och Mac.
- Lägg mappen där du hittar den igen.
- Prompten ska sluta med Pythonkurs respektive pythonkurs.
- Här sparar du all kod under kursen.

2. Skapa projektet

```
uv init --no-package --python 3.13
```

- **--no-package** lägger .py-filer direkt i mappen. Utan flaggan får du en src/-mapp. Den behövs inte.
- **--python 3.13** sätter lägstanivån. Nyare går bra.
- uv skapar pyproject.toml, main.py, .python-version, README.md och .gitignore.
- uv sätter också upp git i mappen. Mer om git i lektion 3.

3. Kör en gång så att .venv skapas

```
uv run main.py
```

- Första uv run laddar hem Python om det behövs, skapar **.venv** och kör filen.
- Du ska se **Hello from pythonkurs!**
- .venv skapas inte av uv init. Den kommer vid uv run, uv sync eller uv add.
- VS Code behöver .venv i nästa steg.

4. Öppna mappen i VS Code

- File, Open Folder, välj C:\Kod\Pythonkurs eller ~/kod/pythonkurs.
- Filträdet ska visa main.py och pyproject.toml.
- Frågar VS Code om du litar på filerna: svara ja. Det är din mapp.

5. Välj rätt Python

- Öppna main.py.
- Klicka på Python-versionen **nere till höger**, eller Ctrl + Shift + P, Python: Select Interpreter.
- **Windows:** .venv\Scripts\python.exe
- **Linux / Mac:** ./.venv/bin/python
- Den är oftast redan märkt Recommended.
- Syns ingen .venv: kör uv run main.py, Refresh, eller Developer: Reload Window.

6. Kör kod från VS Code

- **Run-knappen** uppe till höger när en .py-fil är öppen.
- **Terminal, New Terminal.** Kör med uv run din_fil.py.
- En Python-prompt får du med uv run python.
- Run, debug och terminalen använder nu projektets .venv.

7. Lägga till och ta bort paket

```
uv add pytest
```

- uv add skriver in paketet i pyproject.toml och installerar det.
- uv remove pytest tar bort det igen.
- uv tree visar vad som är installerat.
- Kör alltid programmet med **uv run**, så miljön stämmer med pyproject.toml.

Verifiera att uv och VS Code är samma miljö

```
uv run python -c "import sys; print(sys.executable)"
```

- Sökvägen som skrivs ut ska innehålla projektmappen och **.venv**.
- Gör den det är du klar med det här avsnittet.

Projektet — vanliga fel

- **Ingen .venv i listan:** uv run main.py, Refresh, annars Reload Window.
- **ModuleNotFoundError fast du kört uv add:** du körde vanlig Python. Använd uv run, och kolla steg 5.
- **uv känns inte igen i VS Code:** starta om VS Code helt efter uv-installationen.
- **Fel mapp:** cd till projektmappen. Prompten ska sluta med pythonkurs.
- **En src/-mapp dök upp:** du glömde --no-package. Rensa och kör uv init igen.

Två specialfall senare i kursen

- **Lösa script utan projektfil:** uv venv skapar .venv. uv run din_fil.py fungerar också utan pyproject.toml.
- **En mapp som redan har pyproject.toml**, till exempel inlämningsuppgiften: kör **inte** uv init.
- Där kör du **uv sync**. uv run gör samma sak automatiskt.

JupyterLab

Väg A är den vi använder. Väg B är VS Code.

Vad du ska ha klart

- uv, Python 3.13 och ett projekt med .venv (föregående avsnitt).
- VS Code med Jupyter-extensionen, om du tänker använda Väg B.
- Öppna terminalen i VS Code: Terminal, New Terminal.
- Kommandona är likadana på Windows och Linux/Mac.

Väg A — så gör vi på kursen

- JupyterLab startar från terminalen och öppnas i webbläsaren.
- Samma kommando som lärarens just lab.
- Ha den här vägen igång även om du också använder VS Code.

A1. Kontrollera projektet

```
# Windows
cd C:\Kod\Pythonkurs
dir

# Linux / Mac
cd ~/kod/pythonkurs
ls -a
```

- Gå till mappen du skapade.
- Du ska se pyproject.toml, main.py och **.venv**.
- Saknas .venv: kör uv run main.py en gång.

A2. Lägg notebooks i projektmappen

- JupyterLab visar filerna i mappen du startar det från.
- Utdelade övningar är .ipynb-filer. Dra in dem i projektmappen först.

A3. Starta JupyterLab

```
uv run --with jupyter jupyter lab
```

- --with jupyter lånar in Jupyter för just den här körningen.
- Vill du ha det kvar: uv add jupyter. Sedan räcker uv run jupyter lab.
- Webbläsaren öppnas på http://localhost:8888/lab.
- Gör den inte det: kopiera http://localhost:8888/lab?token=… från terminalen.

A4. Öppna eller skapa en notebook

- Dubbelklicka på en .ipynb till vänster.
- Ny notebook: File, New, Notebook.
- Döp om via högerklick på fliken.

A5. Välj kärna

- JupyterLab frågar efter kernel när notebooken öppnas.
- Välj **Python 3 (ipykernel)**. Det är projektets .venv, eftersom du startade via uv run.
- Byt senare uppe till höger, eller Kernel, Change Kernel.

A6. Kör din första cell

```
print("Hello from Jupyter")
1 + 1
```

- Klicka i cellen och skriv de två raderna.
- **Shift + Enter** kör cellen.
- Du ska se Hello from Jupyter, och sedan 2.
- Sista radens värde visas automatiskt. En ny cell skapas under.

A7. Stäng av

```
uv run --with jupyter --with pyyaml jupyter lab
```

- Notebooken slutar fungera när servern stängs.
- **Ctrl + C** två gånger i terminalen, eller File, Shut Down i JupyterLab.
- Saknas ett paket: stäng av och starta om med --with, eller uv add paketnamn.

Väg B — notebooken inne i VS Code

- Extensionen Jupyter ska redan vara installerad.
- Gör Väg A först, så JupyterLab fungerar. Väg B är ett alternativ till vardags.

B1. Installera Jupyter i projektet

```
uv add jupyter
```

- uv skriver in Jupyter i pyproject.toml.
- Första gången tar en stund. Kommandot ska sluta utan röd text.

B2–B3. Öppna mappen och en notebook

- File, Open Folder, välj projektmappen.
- Högerklick i filträdet, New File, döp till prov.ipynb.
- Eller dra in en utdelad .ipynb och dubbelklicka.

B4. Välj kärna i VS Code

- Uppe till höger: **Select Kernel**.
- Python Environments, den som har **.venv** i sökvägen.
- Frågar VS Code om ipykernel: säg **ja** och vänta tills det är klart.
- Syns ingen .venv: uv add jupyter, sedan Developer: Reload Window.

B5. Kör en cell

- Skriv print("Hello from Jupyter") i cellen.
- **Shift + Enter**.

Två sorters celler

- **Code** — Python som körs. Markera cellen och tryck **Y** (inte medan du redigerar).
- **Markdown** — text och rubriker. Tryck **M**.
- **Shift + Enter** kör och går vidare. **Ctrl + Enter** kör och stannar.
- Kör **uppifrån och ner**. En cell längre ner kan bero på en variabel högre upp.
- Använd inte Run All på övningar. Vissa celler ska fela med flit. En cell i taget.

Kärnan kommer ihåg

- Kärnan är Python-processen. Den minns variabler från tidigare celler.
- **Restart** (rund pil) glömmer allt. Kör sedan från början.
- **Interrupt** (stopp) avbryter en cell som fastnat.
- Ska en notebook lämnas in: rensa output. JupyterLab: Edit, Clear Outputs of All Cells. VS Code: Jupyter: Clear All Outputs.

Tangentbord i notebooken

- Klicka i vänstermarginalen så cellen får en kant. Då gäller kortkommandona.
- **Enter** redigera. **Esc** sluta redigera.
- **A** ny cell ovanför. **B** ny cell under.
- **D D** ta bort cellen. **Z** ångra.
- **M** markdown. **Y** kod. **Shift + Enter** kör.

Jupyter — vanliga fel, 1

- **jupyter: command not found:** du glömde --with jupyter. Kör hela raden.
- **Webbläsaren når inte sidan:** servern är inte igång. Starta om och klistra in token-adressen.
- **ModuleNotFoundError:** Väg A, starta om med --with paketnamn. Väg B, uv add, sedan Restart.
- **Cellen står på [*]** och blir aldrig klar: Interrupt, annars Restart.

Jupyter — vanliga fel, 2

- **Select Kernel visar ingen .venv:** uv add jupyter, sedan Reload Window.
- **VS Code frågar om ipykernel om och om igen:** säg ja och vänta. Välj kärnan igen efteråt.
- **Allt är rött efter Run All:** kör en cell i taget. Vissa celler ska fela.
- jupyter.org/try-jupyter är bara för att känna på gränssnittet. Kursens övningar körs lokalt.

Hello world

När det här fungerar är verktygen ihopkopplade

Förkrav

- Projektmapp med pyproject.toml och .venv.
- Mappen är öppnad i VS Code via File, Open Folder.
- Nere till höger står .venv eller Python 3.13.
- Kommandona är likadana på Windows och Linux/Mac.

1. Skapa hello.py

- Håll musen över filträdet, **New File**, eller File, New File.
- Döp filen till **hello.py**.
- Ändelsen .py betyder Python-kod.

2. Skriv koden och spara

```
print("Hello world!")
```

- print skriver ut det som står i parentesen.
- Texten måste stå inom citationstecken.
- Spara med **Ctrl + S**. Mac: Cmd + S.
- En osparad fil har en fylld prick på fliken.

3A. Kör med Run-knappen

- Klicka på **triangeln** uppe till höger, Run Python File.
- En terminal öppnas nertill och kör filen.
- Prova gärna också vägen via terminalen, en gång var.

3B. Kör från terminalen

```
# Windows, om du står fel
cd C:\Kod\Pythonkurs

# Linux / Mac
cd ~/kod/pythonkurs

uv run hello.py
```

- Terminal, New Terminal.
- Prompten ska sluta med pythonkurs. Annars cd till mappen.
- uv run använder projektets Python. Första gången kan ta en stund.

4. Det du ska se

```
Hello world!
```

- Raden **Hello world!** i terminalen.
- Ser du den är allt som behövs för dag 1 på plats.

5. Ändra och kör igen

```
namn = "Johan"
print("Hello", namn)

# skriver ut
Hello Johan
```

- Byt ut raden, spara, kör igen.
- Byt Johan mot ditt eget namn och kör en gång till.
- Vad namn = … betyder tar vi på lektion 1. Här räcker det att ändringen syns.

Hello world — vanliga fel, 1

- **uv känns inte igen:** starta om terminalen och VS Code. Annars tillbaka till installationsavsnittet.
- **Hittar inte hello.py:** du står i fel mapp, eller filen heter något annat. dir eller ls ska visa hello.py.
- **SyntaxError:** jämför tecken för tecken. Oftast ett saknat citationstecken eller en parentes.

Hello world — vanliga fel, 2

- **NameError: name 'Hello' is not defined:** texten saknar citationstecken. print("Hello world!")
- **Inget händer:** spara med Ctrl + S. Klicka i hello.py så den fliken är aktiv. Kör igen.
- **Run-knappen använder fel Python:** klicka nere till höger och välj .venv.

Nästa steg

- Läs avsnittet om AI-verktyg innan lektion 1.
- Vill du vidare kan du börja på övningarna till lektion 1 när de lagts ut.

8

AI-verktyg

Tillåtna i kursen. Inte på proven. Avstängda i editorn nu.

Kort version

- **Du får använda** chattbotar och kodassistenter. Det hör till yrket.
- **Du ansvarar** för all kod du lämnar in, även den en AI föreslog.
- Kan du inte förklara en rad ska den inte vara med.
- **Stäng av** komplettering i editorn i början. Slå på den när du står stadigare.
- **På veckoprov och tentor är AI inte tillåtet**, om inget annat sägs.

Vad de är bra på

- Förklara ett felmeddelande. Klistra in felet och raderna det pekar på.
- Förklara en rad, en funktion eller ett begrepp du inte förstår.
- Slå upp syntax: hur läser jag en textfil rad för rad i Python 3.13?
- Skriva rutinkod du redan kan, till exempel ett test till din egen funktion.
- Bolla upplägg, och föreslå namn, struktur och kommentarer.

Vad du ska vara vaksam på

- **Självsäkert fel.** Det låter lika säkert när det har fel. Kör koden. Läs dokumentationen.
- **Föråldrat.** Python 2, %-formatering, os.path i stället för pathlib.
- **För mycket kod.** Hela lösningen på en gång. Då lär du dig inget, och du kan inte felsöka.
- **Påhittade funktioner** som låter rimliga men inte finns.
- **Sekretess.** Klistra aldrig in riktiga loggar, nycklar, lösenord eller personuppgifter. Hitta på data.

Ett exempel på ett självsäkert fel

- Fråga: hur kollar jag om en fil finns?
- Svar: os.path.isfile(path), eller den nyare os.exists(path).
- os.path.isfile finns. **os.exists finns inte.** Det heter os.path.exists. pathlib har Path(path).exists().
- Köper du svaret rakt av får du AttributeError, och ett fel du inte förstår.
- Rätt reaktion: testa med uv run python, eller slå upp os och pathlib i dokumentationen.

Så lär du dig något av svaret

- **1.** Försök själv först. Skriv ett utkast, eller skriv ner exakt var du fastnat.
- **2.** Fråga smått. Ett tips, inte hela lösningen.
- **3.** Skriv av för hand. Kör. Fråga vidare tills du fattar.
- **4.** Verifiera mot dokumentationen, help(…) i REPL, eller att koden gör rätt på din indata.
- **5.** Vad lärde du dig, och vad gör du annorlunda nästa gång?

Checklista innan svaret får ligga kvar

- Jag kan förklara varje rad med egna ord.
- Jag har kört koden, även på ett kantfall.
- Inga påhittade funktioner. Det jag anropar finns i dokumentationen.
- Stilen matchar kursen: f-strängar, pathlib, Python 3.13 eller senare.
- Jag har inte klistrat in känslig data för att få fram svaret.
- Kan du inte bocka alla rutorna — använd inte koden ännu.

Fråga smått

- Mindre bra: ”Lös uppgift 2 åt mig.”
- Bättre: ”Vad betyder KeyError: 'time' på rad 12?”
- Mindre bra: ”Skriv en loggparser.”
- Bättre: ”Jag har en lista med rader. Hur delar jag varje rad på första kolontecknet?”
- Mindre bra: ”Varför funkar inte min kod?” utan att visa koden.
- Bättre: funktionen och felmeddelandet, sedan frågan.

Prov och inlämning

- På veckoprov och tentor: inga chattbotar, kodassistenter eller AI-komplettering.
- I inlämningsuppgiften får du använda verktygen.
- Du ska kunna förklara varje del muntligt.
- Skriv en mening i README, eller överst i filen, om vad AI användes till.
- En lösning du inte kan förklara godkänns inte.

Stäng av kompletteringen nu

- Förslag på hela rader gör att grunderna inte fastnar.
- Logga inte in på Copilot när VS Code frågar.
- Ctrl + komma, sök inline suggest, stäng av Editor: Inline Suggest: Enabled.
- Samma steg som i VS Code-avsnittet.
- Vi tar upp det igen på lektion 1 och i kursavslutningen.

Klart

Du kan köra ett Python-program. Ta med det som strulade till lektion 1.